

MouseMine Manual Build Guide

Step-by-step guide for running a MouseMine build from inside the existing Docker container on AllianceMineDev (172.31.60.197).

Container Overview

Property	Value
Container	mousemine
Image	mousemine:latest
OS	Ubuntu 20.04
Java	OpenJDK 11
Build tool	Apache Ant 1.10.7
Local PostgreSQL	Running inside container, port 5432
Data mount	/data → ~/agr_mousemine/mousemine_data on host
ETL	/mousemine_etl — Python-based data extraction
Bio sources	/mousemine_bio_sources_mgi
Mine code	/mousemine and /intermine

Key Directories

```
/mousemine/          # Mine project (project.xml, build.xml)
/intermine/          # InterMine core
/mousemine_bio_sources_mgi/ # MGI-specific bio sources
/mousemine_etl/      # ETL pipeline (Python data extractors)
/mousemine_etl/bin/   # ETL scripts
/mousemine_etl/bin/config.cfg # ETL data source configuration
/data/               # Mounted data directory
/root/.intermine/     # Properties file location
```

Prerequisites

1. SSH into AllianceMineDev

```
ssh -i ~/.ssh/AGR-ssl3.pem ec2-user@172.31.60.197
```

2. Enter the container

```
docker exec -it mousemine bash
```

3. Create the properties file

The properties file tells MouseMine which databases to use. To use **RDS** instead of the local PostgreSQL:

```
mkdir -p /root/.intermine
cat > /root/.intermine/mousemine.properties << 'EOF'
# MouseMine Properties — RDS Connection

# Production Database
db.production.datasource.serverName=intermine-postgres.cmnnhlso7wdi.us-east-
↪ 1.rds.amazonaws.com
db.production.datasource.databaseName=mousemine_db
db.production.datasource.user=postgres
db.production.datasource.password=<RDS_PASSWORD>
db.production.datasource.maxConnections=20
db.production.driver=org.postgresql.Driver
```

```

db.production.platform=PostgreSQL

# Items Database (staging for data integration)
db.common-tgt-items.datasource.serverName=intermine-postgres.cmnhlso7wdi.us-east-
↳ 1.rds.amazonaws.com
db.common-tgt-items.datasource.databaseName=mousemine_items
db.common-tgt-items.datasource.user=postgres
db.common-tgt-items.datasource.password=<RDS_PASSWORD>
db.common-tgt-items.datasource.maxConnections=10
db.common-tgt-items.driver=org.postgresql.Driver
db.common-tgt-items.platform=PostgreSQL

# User Profile Database
db.userprofile-production.datasource.serverName=intermine-postgres.cmnhlso7wdi.us-east-
↳ 1.rds.amazonaws.com
db.userprofile-production.datasource.databaseName=mousemine_profiles_db
db.userprofile-production.datasource.user=postgres
db.userprofile-production.datasource.password=<RDS_PASSWORD>
db.userprofile-production.datasource.maxConnections=10
db.userprofile-production.driver=org.postgresql.Driver
db.userprofile-production.platform=PostgreSQL

# Webapp
project.title=MouseMine
project.subTitle=An integrated data warehouse for mouse genomics
project.releaseVersion=1.8
webapp.deploy.url=http://localhost:8080
webapp.baseurl=http://localhost:8080
webapp.path=mousemine
webapp.manager=manager
webapp.password=manager

# SuperUser
superuser.account=superuser@mail_account

# Solr (local to container or external)
index.solrurl=http://localhost:8983/solr/mousemine-search
autocomplete.solrurl=http://localhost:8983/solr/mousemine-autocomplete
EOF

```

Replace <RDS_PASSWORD> with the actual RDS password (same one used for AllianceMine builds, found in the AllianceMine .env file).

To use the **local PostgreSQL** instead (already running in the container):

```

cat > /root/.intermine/mousemine.properties << 'EOF'
# MouseMine Properties – Local PostgreSQL

db.production.datasource.serverName=localhost
db.production.datasource.databaseName=mousemine
db.production.datasource.user=mousemine
db.production.datasource.password=mousemine
db.production.datasource.maxConnections=20
db.production.driver=org.postgresql.Driver
db.production.platform=PostgreSQL

db.common-tgt-items.datasource.serverName=localhost
db.common-tgt-items.datasource.databaseName=items
db.common-tgt-items.datasource.user=mousemine

```

```

db.common-tgt-items.datasource.password=mousemine
db.common-tgt-items.datasource.maxConnections=10
db.common-tgt-items.driver=org.postgresql.Driver
db.common-tgt-items.platform=PostgreSQL

db.userprofile-production.datasource.serverName=localhost
db.userprofile-production.datasource.databaseName=userprofile
db.userprofile-production.datasource.user=mousemine
db.userprofile-production.datasource.password=mousemine
db.userprofile-production.datasource.maxConnections=10
db.userprofile-production.driver=org.postgresql.Driver
db.userprofile-production.platform=PostgreSQL

project.title=MouseMine
project.subTitle=An integrated data warehouse for mouse genomics
project.releaseVersion=1.8
webapp.deploy.url=http://localhost:8080
webapp.baseurl=http://localhost:8080
webapp.path=mousemine
webapp.manager=manager
webapp.password=manager
superuser.account=superuser@mail_account
index.solrurl=http://localhost:8983/solr/mousemine-search
autocomplete.solrurl=http://localhost:8983/solr/mousemine-autocomplete
EOF

```

4. Create RDS databases (if using RDS)

From the host machine or any machine with psql:

```

export PGPASSWORD='<RDS_PASSWORD>'
PSQL="psql -h intermine-postgres.cmnhlso7wdi.us-east-1.rds.amazonaws.com -U postgres -d
↳ postgres"

$PSQL -c "CREATE DATABASE mousemine_db"
$PSQL -c "CREATE DATABASE mousemine_items"
$PSQL -c "CREATE DATABASE mousemine_profiles_db"

```

Running the ETL (Data Extraction)

The ETL extracts data from MGI, downloads ontologies, UniProt, InterPro, etc.

```

cd /mousemine_etl
python3 bin/refresh.py

```

This reads bin/config.cfg and downloads/processes each data source into /var/lib/jenkins/etl_build/etl/output/ (the path referenced in project.xml as &mm_data;).

Check the ETL output directory exists and has data:

```
ls /var/lib/jenkins/etl_build/etl/output/
```

If this path doesn't exist, you may need to create it and symlink:

```

mkdir -p /var/lib/jenkins/etl_build/etl/output
# or symlink from wherever the ETL writes to:
# ln -s /actual/output/path /var/lib/jenkins/etl_build/etl/output

```

Running the Build

All build commands run from inside the container at /mousemine:

```
cd /mousemine
```

Step 1: Build the database schema

```
ant -Drelease=1.8 -v build-db
```

Step 2: Integrate data sources (project_build)

The project_build Perl script processes each source in project.xml:

```
# Full build from scratch
/intermine/project_build -b -E UTF8 localhost /data/dump
```

```
# Resume from last checkpoint
/intermine/project_build -l -E UTF8 localhost /data/dump
```

Flags: - -b — build database first (fresh build only) - -l — load last dump and resume (after failure) - -E UTF8 — required if using RDS (default SQL_ASCII breaks createdb on RDS) - localhost — dump host (where pg_dump runs) - /data/dump — dump file prefix

This takes several hours.

Step 3: Post-processing

```
ant -v postprocess
```

Step 4: Build user profile database

```
ant -v build-userdb
```

Step 5: Build WAR file

```
ant -v war
```

Step 6: Deploy (optional)

```
ant -v deploy
```

Quick Reference: Single Build Commands

```
# Inside the container:
docker exec -it mousemine bash
cd /mousemine
```

```
# Fresh build (all steps)
ant -Drelease=1.8 -v build-db
/intermine/project_build -b -E UTF8 localhost /data/dump
ant -v postprocess
ant -v build-userdb
ant -v war
```

```
# Resume after failure
/intermine/project_build -l -E UTF8 localhost /data/dump
ant -v postprocess
```

Monitoring

```
# Check what source is being integrated
tail -f /data/dump/pbuild.log

# Check database size (local PostgreSQL)
su - postgres -c "psql -c \"SELECT pg_size_pretty(pg_database_size('mousemine'))\""

# Check database size (RDS)
PGPASSWORD='...' psql -h intermine-postgres.cmnhlso7wdi.us-east-1.rds.amazonaws.com \
-U postgres -d postgres \
-c "SELECT pg_size_pretty(pg_database_size('mousemine_db'))"
```

Differences from AllianceMine Build

Feature	MouseMine	AllianceMine
Build tool	Ant	Gradle
Java	11	8
Data extraction	ETL (Python, mousemine_etl)	FMS API + S3 (extract_data.py)
PostgreSQL	Local (in container) or RDS	RDS only
Orchestration	Manual (ant commands)	Automated (build_full.py)
Source repos	MGI-specific bio sources	Alliance bio sources
Container	All-in-one (build + DB + Tomcat)	Build-only (external RDS + Tomcat)
Data sources config	project.xml uses &mm_data; entity	project.xml uses absolute paths
project_build location	/intermine/project_build	/root/alliancemine/project_build

Troubleshooting

“No such file” for data sources

The project.xml uses &mm_data; which resolves to /var/lib/jenkins/etl_build/etl/output. If this directory doesn't exist or has no data, run the ETL first (python3 bin/refresh.py from /mousemine_etl).

SQL_ASCII encoding error with RDS

Use -E UTF8 flag with project_build. RDS requires UTF8 encoding for CREATE DATABASE.

Can't connect to RDS from inside container

The container needs network access to the RDS endpoint. Check:

```
pg_isready -h intermine-postgres.cmnhlso7wdi.us-east-1.rds.amazonaws.com -p 5432
```

If it fails, the container may need to be on the right Docker network or the security group may need updating.

Out of memory

MouseMine runs on Java 11 with whatever the container's default heap is. Set memory if needed:

```
export ANT_OPTS="-Xmx8g"
```